



PUC Minas

Arquitetura de Computadores III

Introdução a Processadores Paralelos

Cenário Ideal

- Criar computadores poderosos simplesmente conectando muitos processadores menores já existentes.
- Onde podemos comprar quantos processadores puder e receber desempenho proporcional
- Softwares devem funcionar com número variável de processadores.

Introdução

Eficiência Energética e Multicore

- **O Problema da Energia**
 - Consumo energético tornou-se fator crítico
 - Limitações de aumento de frequência de clock
 - Processadores menores podem ser mais eficientes
 - Melhor desempenho por Joule
- **Motivação moderna**
 - Não é apenas desempenho
 - Também é eficiência energética

Introdução

Disponibilidade e Robustez

- Software multiprocessado deve escalar e também tolerar falhas.
- Exemplo: Se 1 processador falha em um sistema com n processadores
 - Então sistema continua com $n - 1$ processadores.
- Multiprocessadores melhoram a disponibilidade

Introdução

Tipos de Paralelismo

Tipo	Descrição	Exemplo
Paralelismo de tarefas (nível de processo)	Múltiplas tarefas independentes rodando em paralelo	Vários aplicativos single-thread
Programa de processamento paralelo	Um único programa rodando em múltiplos processadores simultaneamente	Simulações científicas

Introdução

Aplicações e Mudança de Paradigma

- Problemas científicos antigos justificaram computadores paralelos inovadores.
- Atualmente clusters de microprocessadores resolvem esses problemas e também outras aplicações do cotidiano:
 - Mecanismos de busca
 - Servidores Web e de e-mail
 - Bancos de dados

Introdução

O Impacto da Energia

- Aumento de desempenho não virá mais de:
 - Clock muito mais alto
 - CPI (ciclos por instrução) drasticamente melhorado
- Múltiplos núcleos (cores) no mesmo chip → micro processadores multicore.
- Cores = processadores em um chip multicore.
- Arquitetura predominante **SMP** (*Shared Memory Processors*), onde todos cores tem acesso ao mesmo espaço de endereçamento físico.

Introdução

- **O Grande Desafio**

- *"O estado da tecnologia hoje significa que programadores que se importam com desempenho devem se tornar programadores paralelos."*

- **Desafio central**

- Criar *hardware* e *software* que torne fácil escrever programas paralelos corretos, eficientes em desempenho e energia, à medida que o número de núcleos por *chip* aumenta.

Dificuldade da Programação Paralela

Por que programas paralelos são difíceis?

- Hardware paralelo já está amplamente disponível
- O grande desafio é o software
- Poucos programas conseguem usar muitos processadores eficientemente
- Complexidade aumenta conforme cresce o número de núcleos

Dificuldade da Programação Paralela

Primeiras razões da dificuldade

1. Expectativa de ganho: Sem melhoria real, o programa sequencial é mais simples.
2. Técnicas uniprocessador já ajudam (sem esforço do programador):
 - Superscalar
 - Execução fora de ordem
 - Aproveitam paralelismo em nível de instrução (ILP)
 - Programas sequenciais rodam mais rápido sozinhos.

Consequência: Menos pressão para reescrever programas para multiprocessadores.

Dificuldade da Programação Paralela

A Analogia dos 8 Jornalistas

- **Objetivo:** Escrever uma única história 8x mais rápido.
- **Desafios:**
 - Dividir em 8 partes iguais (senão, alguns ficam ociosos)
 - Evitar tempo excessivo em comunicação em vez de escrever
- Tradução para programação paralela:
 - Scheduling (escalonamento)
 - Particionamento do trabalho
 - Balanceamento de carga
 - Tempo de sincronização
 - Overhead de comunicação
- Quanto mais processadores (ou jornalistas), maior o desafio.

Dificuldade da Programação Paralela

Desafio do speedup: Exemplo 1

- Suponha que você deseja atingir um speedup de 90 vezes utilizando 100 processadores. Qual o percentual de computação que pode ser sequencial?
- Vamos usar a Lei de Amdahl:

$$Speedup = \frac{1}{(1 - Fração_{Paralela}) + \frac{Fração_{Paralela}}{100}}$$

Dificuldade da Programação Paralela

Desafio do speedup: Exemplo 1

- Suponha que você deseja atingir um speedup de 90 vezes utilizando 100 processadores. Qual o percentual de computação que pode ser sequencial?
- Vamos usar a Lei de Amdahl:

$$90 = 1 / ((1 - \textit{Fração}_{paralela}) + (\textit{Fração}_{paralela}/100))$$

$$\textit{Fração}_{paralela} = 0,999$$

- Ou seja a parte sequencial da computação precisar ser apenas de 0,1%

Dificuldade da Programação Paralela

Desafio do speedup: Exemplo 2

- Realizar a soma de 10 escalares, e de uma matriz 10×10
- O tempo de cada soma é $t_{\text{soma}} = 1$
- Apenas as operações sobre matrizes são paralelizáveis
- Calcule o speedup e a eficiência para 10 e 40 processadores.

Dificuldade da Programação Paralela

Desafio do speedup: Exemplo 2

- Realizar a soma de 10 escalares, e de uma matriz 10×10
- O tempo de cada soma é $t_{\text{soma}} = 1$
- Apenas as operações sobre matrizes são paralelizáveis
- Calcule o speedup para 10 e 40 processadores.
 - Para 1 processador: $(10 \times 1) + (100/1) \times 1 = 110$
 - Para 10 processadores: $(10 \times 1) + (100/10) \times 1 = 20$
 - Para 40 processadores: $(10 \times 1) + (100/40) \times 1 = 12,5$
- **Speedup:**
 - Para 10 processadores: $110/20 = 5,5$
 - Para 40 processadores: $110/12,5 = 8,8$
- **Eficiência:**
 - Para 10 processadores: $5,5/10 = 55\%$
 - para 40 processadores: $8,8/40 = 22\%$

Dificuldade da Programação Paralela

Desafio do speedup: Exemplo 3

- Realizar a soma de 10 escalares, e de uma matriz 20×20
- O tempo de cada soma é $t_{\text{soma}} = 1$
- Apenas as operações sobre matrizes são paralelizáveis
- Calcule o speedup e a eficiência para 10 e 40 processadores.

Dificuldade da Programação Paralela

Desafio do speedup: Exemplo 3

- Realizar a soma de 10 escalares, e de uma matriz 20×20
- O tempo de cada soma é $t_{\text{soma}} = 1$
- Apenas as operações sobre matrizes são paralelizáveis
- Calcule o speedup para 10 e 40 processadores.
 - Para 1 processador: $(10 \times 1) + (400/1) \times 1 = 410$
 - Para 10 processadores: $(10 \times 1) + (400/10) \times 1 = 50$
 - Para 40 processadores: $(10 \times 1) + (400/40) \times 1 = 20$
- **Speedup:**
 - Para 10 processadores: $410/50 = 8,2$
 - Para 40 processadores: $410/20 = 20,5$
- **Eficiência:**
 - Para 10 processadores: $8,2/10 = 82\%$
 - para 40 processadores: $20,5/40 = 51,25\%$

Dificuldade da Programação Paralela

Escala Forte (Strong Scaling)

- **Definição**
 - Tamanho do problema permanece fixo
 - Aumenta número de processadores
- **Objetivo**
 - Resolver o mesmo problema mais rapidamente.
- **Dificuldade**
 - Fortemente limitado pela Lei de Amdahl.

Dificuldade da Programação Paralela

Escala Fraco (Weak Scaling)

- **Definição**
 - Problema cresce junto com número de processadores
- **Objetivo**
 - Manter mesma carga por processador.
- **Vantagem**
 - Melhor aproveitamento do paralelismo.

Dificuldade da Programação Paralela

Cache Também Influencia

- Problema
 - Weak scaling pode degradar desempenho se:
 - Dados deixam de caber em cache
 - Acessos à memória aumentam
- Consequência
 - Mais processadores nem sempre ajudam.

Dificuldade da Programação Paralela

Balanceamento é Fundamental

- **Se cargas forem desiguais:**
 - Alguns núcleos ficam ociosos
 - Eficiência cai
 - Escalabilidade piora
- **Objetivo**
 - Maximizar utilização dos recursos.

Dificuldade da Programação Paralela

Impacto do balanceamento no speedup

- Para alcançar um *speedup* de 20,5 no exemplo 3, assumimos que a carga está perfeitamente balanceada, ou seja, cada um dos 40 processadores faz 2,5% do trabalho
- Qual seria o resultado se um dos processadores fizer 5% do trabalho (20 somas) ?

Dificuldade da Programação Paralela

Impacto do balanceamento no speedup

- Para alcançar um *speedup* de 20,5 no exemplo 3, assumimos que a carga está perfeitamente balanceada, ou seja, cada um dos 40 processadores faz 2,5% do trabalho
- Qual seria o resultado se um dos processadores fizer 5% do trabalho (20 somas) ?
 - Para 40 processadores: $(10 \times 1) + \text{Max}((380/39) \times 1, (20/1) \times 1) = 30$
 - **Speedup:**
 - Para 40 processadores: $410/30 = 14$
 - **Eficiência:**
 - para 40 processadores: $14/40 = 35\%$
- **Speedup** cai de 20,5 para 14 e a **eficiência** de 51,25% para 35%

SISD, MIMD, SIMD, SPMD, e Processadores Vetoriais

Taxonomia de Flynn (anos 60)

- **SISD** (Single Instruction, Single Data)
- **SIMD** (Single Instruction, Multiple Data)
- **MISD** (Multiple Instruction, Single Data)
- **MIMD** (Multiple Instruction, Multiple Data)

		Data Streams	
		Single	Multiple
Instruction Streams	Single	SISD: Intel Pentium 4	SIMD: SSE instructions of x86
	Multiple	MISD: No examples today	MIMD: Intel Core i7

SISD

Single Instruction Single Data

- Características
 - Uma instrução
 - Um fluxo de dados
 - Execução sequencial
- Exemplo
 - Processador uniprocessado tradicional.
- Modelo clássico
 - Von Neumann

MIMD

Multiple Instruction Multiple Data

- **Características**
 - Múltiplos processadores
 - Cada núcleo executa instruções independentes
 - Dados independentes
- **Exemplo**
 - Multicore modernos
 - Clusters
 - Servidores SMP

Modelo de Programação SPMD

Single Program Multiple Data

- **Ideia principal**
 - Um único programa executa em todos os processadores.
- **Diferença**
 - Cada processador:
 - executa partes diferentes
 - usa dados diferentes
- **Controle**
 - Uso de condicionais:
 - if
 - switch
 - rank/thread ID

Forma Mais Comum de Programar MIMD

- **Exemplo**

- MPI/OpenMP:

```
if(rank == 0){  
    processa_A();  
}else{  
    processa_B();  
}
```

- **Característica**

- Mesmo código binário em todos os núcleos.

MISD

Multiple Instruction Single Data

- **Ideia**
 - Múltiplas operações sobre mesmo fluxo de dados.
- **Exemplo aproximado**
 - Pipeline de processamento:
 - parse
 - descritografia
 - descompressão
 - busca
- Raramente encontrado

SIMD

Single Instruction Multiple Data

- **Funcionamento**
 - Uma instrução:
 - controla várias ALUs
 - opera sobre muitos dados simultaneamente
- **Exemplo**
 - Somar 64 números em paralelo.

SIMD e o Paralelismo a Nível de Dados

Paralelismo de Dados

- **SIMD funciona melhor quando:**
 - dados possuem mesma estrutura
 - operações são repetitivas
 - arrays e loops predominam

- **Exemplo típico**

```
for(i=0;i<N;i++)  
    C[i]=A[i]+B[i];
```

Vantagens do SIMD

- **Benefícios**

- Controle centralizado
- Menor bandwidth de instruções
- Menor consumo energético
- Melhor throughput
- Código único

- **Ideia**

- Uma instrução controla muitos elementos.

Problema do SIMD

Divergência de Controle

- **Caso problemático**
 - switch/case ou muitos ifs
- **Situação**
 - Cada ALU deseja executar operação diferente.
- **Consequência**
 - Algumas ALUs ficam desativadas
 - Desempenho cai
 - Desempenho $\approx 1 / n$
 - para n caminhos diferentes.

SIMD no x86

MMX, SSE e AVX

- **Evolução**
 - MMX / SSE / SSE2 / AVX / AVX2
- **Objetivo**
 - Executar múltiplos dados simultaneamente.
- **Exemplo**
 - AVX:
 - 8 operações de precisão dupla simultâneas

Arquiteturas Vetoriais

- **Origem**
 - Seymour Cray na década de 1970
- **Ideia**
 - Operar sobre vetores inteiros de dados.
- **Filosofia**
 - carregar vetores
 - operar em registradores vetoriais
 - armazenar resultados

Registradores Vetoriais

Exemplo

- 32 registradores vetoriais:
 - cada um com 64 elementos de 64 bits
- Benefício
 - Grande quantidade de operações por instrução.

Extensão Vetorial do RISC-V

RISC-V V Extension

- Instruções vetoriais
 - `vfadd.vv`
 - `vfmul.vf`
 - `vle.v`
 - `vse.v`
- Configuração
 - `vsetvli` define:
 - tamanho dos elementos
 - largura vetorial

Exemplo DAXPY

Operação Vetorial

- Fórmula: $Y = a \times X + Y$

RISC-V convencional:

```
fld f0, a(x3)      # load scalar a
addi x5, x19, 512   # end of array X
loop: fld f1, 0(x19) # load x[i]
    fmul.d f1, f1, f0 # a * x[i]
    fld f2, 0(x20)    # load y[i]
    fadd.d f2, f2, f1  # a * x[i] + y[i]
    fsd f2, 0(x20)    # store y[i]
    addi x19, x19, 8   # increment index to x
    addi x20, x20, 8   # increment index to y
    bltu x19, x5, loop # repeat if not done
```

RISC-V Vetorial:

```
fld f0, a(x3)      # load scalar a
vsetvli x0, x0, e64 # 64-bit-wide elements
vle.v v0, 0(x19)    # load vector x
vfmul.vf v0, v0, f0  # vector-scalar multiply
vle.v v1, 0(x20)    # load vector y
vfadd.vv v1, v1, v0  # vector-vector add
vse.v v1, 0(x20)    # store vector y
```

Comparação Assembly Escalar vs Vetorial

Redução de Instruções

- **Código escalar**
 - Mais de 500 instruções dinâmicas
- **Código vetorial**
 - Apenas 7 instruções
- **Benefícios**
 - Menor largura de banda
 - Menor energia
 - Melhor desempenho

Hazards em Arquiteturas Vetoriais

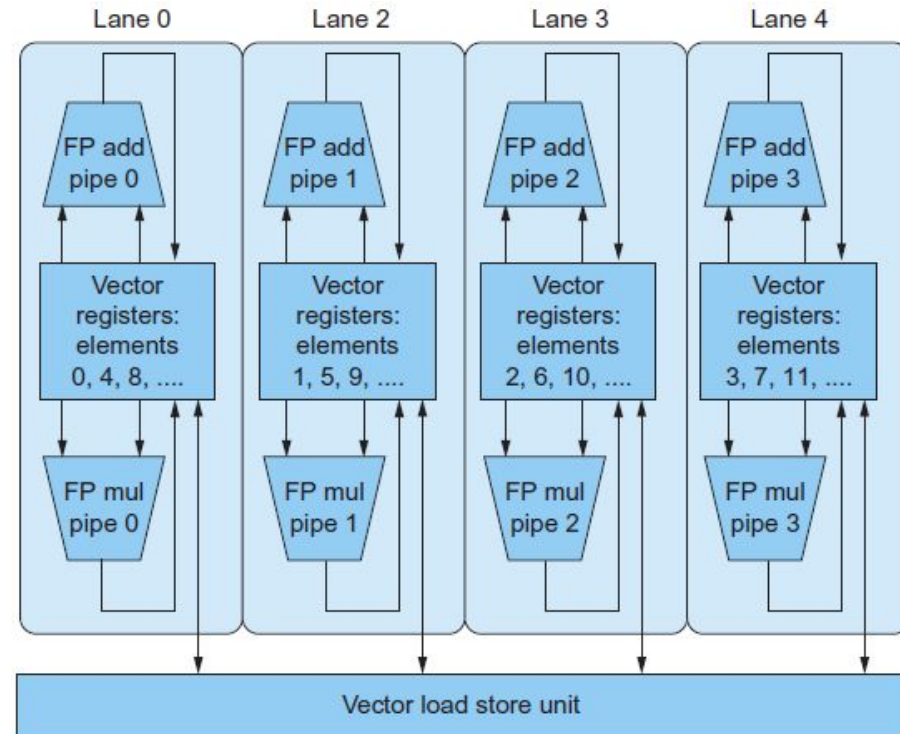
Menos Pipeline Hazards

- **Escalar**
 - Hazards ocorrem:
 - para cada elemento
- **Vetorial**
 - Hazard ocorre:
 - para cada vetor
- **Consequência**
 - Fluxo contínuo no pipeline.

Vector Lanes

Paralelismo Interno

- **Arquiteturas vetoriais modernas** possuem:
 - múltiplos pipelines paralelos
 - vector lanes
- **Objetivo**
 - Produzir vários resultados por ciclo.



Vetorial vs Escalar

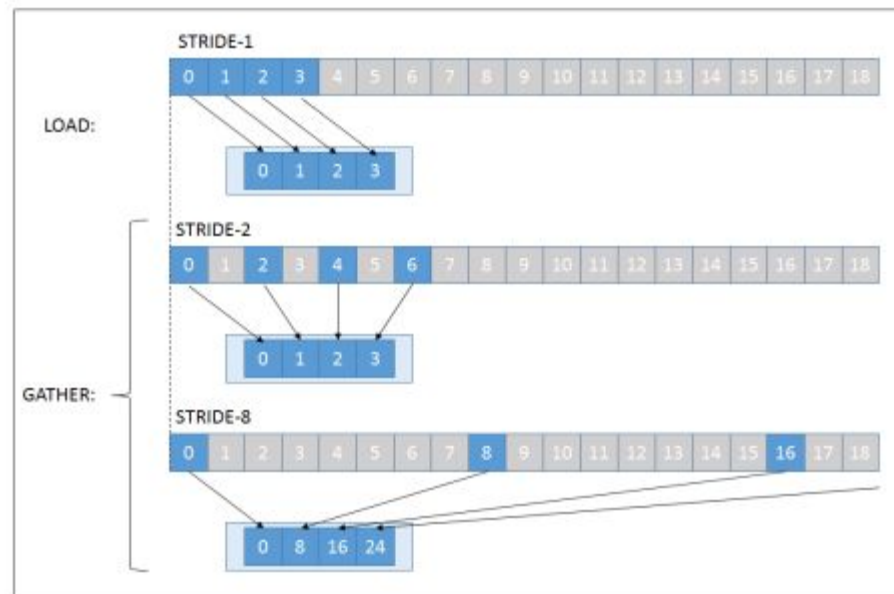
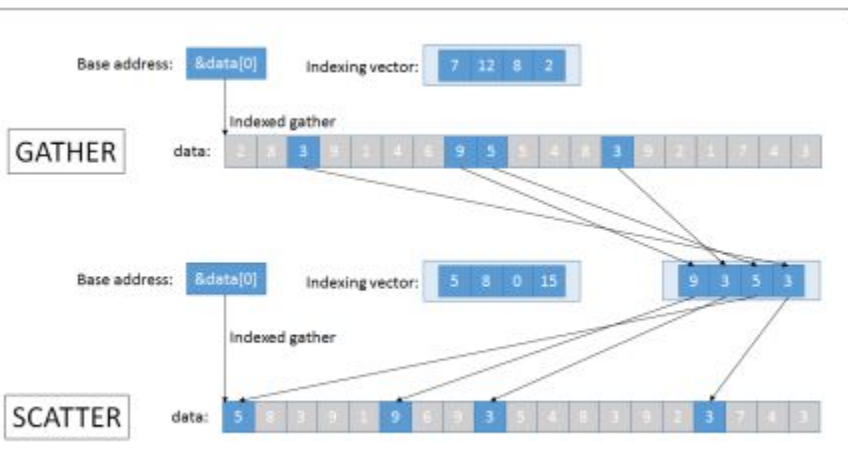
Principais Diferenças

- **Vetorial**
 - uma instrução → muitas operações
 - menos fetch/decode
 - menos hazards
 - acesso eficiente à memória
- **Escalar**
 - operação por operação

Acesso à Memória Vetorial

Padrões de acesso

- Contíguo
- Gather-Scatter
- Strided



Vetorial vs Extensões SIMD

Diferenças Importantes

- **SIMD multimídia**
 - tamanho embutido no opcode
 - AVX, SSE, MMX
- **Vetorial**
 - tamanho definido em registrador
 - mais flexível
 - compatibilidade binária

Escalabilidade das Arquiteturas Vetoriais

Evolução Mais Elegante

- **Vantagens**
 - fácil aumentar largura vetorial
 - não precisa criar centenas de novos opcodes
 - melhor adaptação futura
- **Consequência**
 - Arquiteturas vetoriais:
 - eficientes
 - escaláveis
 - energeticamente vantajosas

Resumo

Conceitos Fundamentais

- SISD → sequencial
- MIMD → múltiplos fluxos independentes
- SPMD → modelo dominante em MIMD
- SIMD → uma instrução para muitos dados
- Vetorial → forma sofisticada de SIMD
- Paralelismo a nível de dados é essencial